Extra documents about Artificial Intelligence

Christian Rinderknecht

19 October 2008

Abstract Neural Networks

In general, all the abstract neural networks (textbook, section 20.5, page 736.) share the following features:

- a set of simple processing nodes (modeling neurons),
- a pattern of connection (edges between abstract neurons),
- a rule for propagating signals through the network (when and which nodes),
- a rule for combining input signals (entering a node),
- a rule for calculating *the* output signal (leaving a node),
- a learning rule to adapt the weights (a floating-point number on an edge).

An abstract neural network can be considered as a real-valued oriented graph.

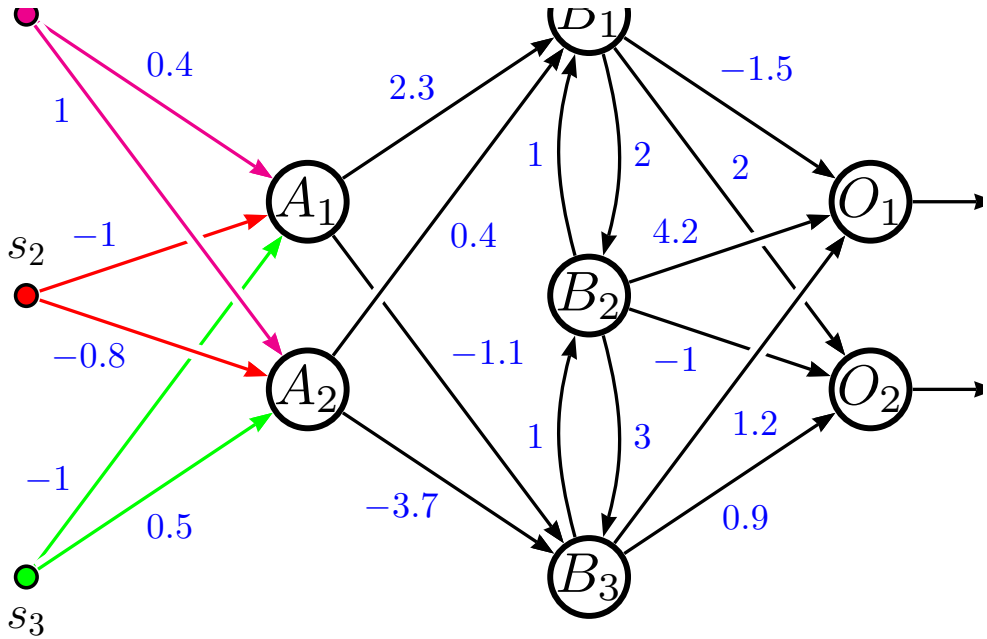
Abstract Neural Networks (cont)

Let us detail those features.

- **Nodes** are of several kinds: **input nodes** and **output nodes**. It may exist, depending on the connectivity, **hidden nodes**, which are neither input nor output nodes.
- **Patterns of connection** characterise the way nodes are interconnected:
  - each node can be connected to every other node;
  - in some other type, nodes may be arranged into a **hierarchy of layers** in which connections are allowed only between nodes in immediately adjacent layers (**feed-forward networks**);
  - another kind allows **feedback connections** between adjacent layers, or within a layer, or allow nodes to send a signal back to themselves.

Abstract Neural Network/Example

An example of a 4-layered network Input signals s_1, s_2, s_3 enter the network at the input layer which is the left column (same color meaning same signal value). The first hidden layer is $\{A_1, A_2\}$, the second one is $\{B_1, B_2, B_3\}$ and the output layer is $\{O_1, O_2\}$. The output signals are going out of O_1 and O_2. Note the hierarchy of layers and the negative weights (in blue).



Abstract Neural Networks (cont)

- **Propagation rules** are rules stating **when nodes are updated**, i.e., when to combine input signals and calculate output signals, and **when they send a signal** to another node. Also rules specify **which nodes are updated**. For instance, a node is selected at random for updating, whereas in another kind, some groups of nodes must be updated before another one.
- **Combination rules** consists in summing the weighted values of incoming signals. If $s_1, s_2, \ldots, s_n$ are the signal values entering a node using edges weighted $w_1, w_2, \ldots, w_n$, then the global input signal s, called **net input**, is

$$s = \sum_{i=1}^{n} w_i \cdot s_i$$

For instance, net input of A_1, in previous example, is $0.4 \times s_1 - s_2 - s_3$.

Abstract Neural Networks (cont)

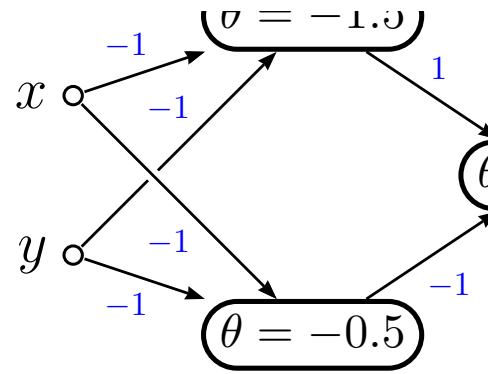
- **Output calculation** is done by a function, called **activation function**, which, given the net input, computes the output signal, called **activation** — which can be either real-valued, e.g. belonging to $[-1, +1]$, or discrete, e.g. in $\{0, 1\}$. Here are some activation functions whose domain is $\mathbb{R}$:
  - **Identity.** It is simply $f(x) = x$.
  - **Binary threshold.** With threshold θ, it is $f_\theta(x) = \begin{cases} 1 & \text{if } x \geqslant \theta \\ 0 & \text{if } x < \theta \end{cases}$
  - **Sigmoid.** This function has a "S" shape (hence its name) and its co-domain (i.e., images set) is the real interval $]0, 1[$:

$$f(x) = \frac{1}{1 + e^{-x}}$$

Example of XOR

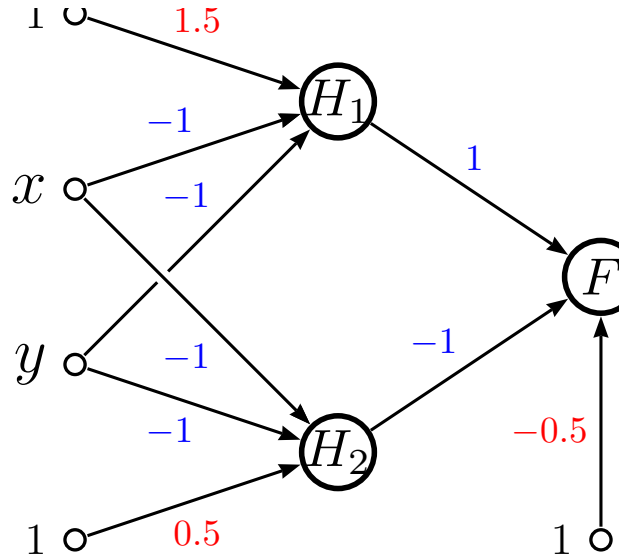
XOR. Here is an example of an extremely simple and useless abstract neural network: how to compute the boolean function XOR (exclusive disjunction), noted $\oplus$. The definition is: $x \oplus y \triangleq x \cdot \overline{y} + \overline{x} \cdot y$, where $+$ is the disjunction (*or*), $\cdot$ is the conjunction (*and*) and $\overline{x}$ is the negation of x.

The input layer provides signals x and y to the hidden layer, which is made of two nodes whose activation functions are binary thresholds of 1.5 and 0.5. The output layer is one state whose threshold is -0.5.



Example of XOR (cont)

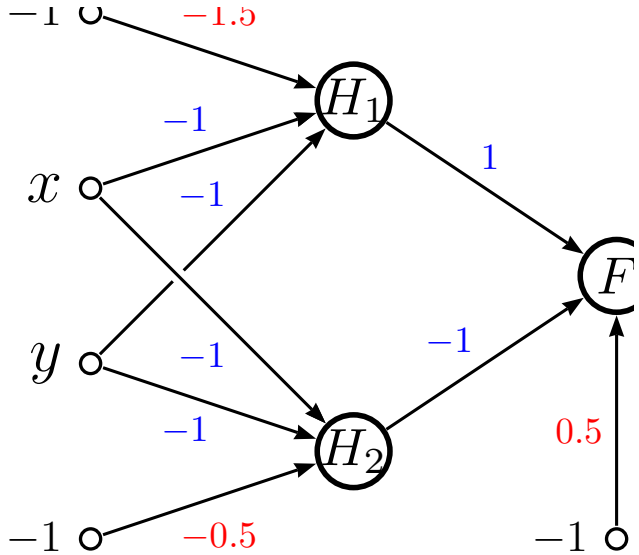
Before verifying the network behaviour, let us note that a node with a binary θ-threshold function is equivalent to the same node with a 0-threshold plus an input signal of 1 with weight $-\theta$ (called **bias**). In other words, the previous example can be rewritten as shown in the facing column (bias in red).



Exercise. Show that this network realises the boolean XOR relationship.

Example of XOR (cont)

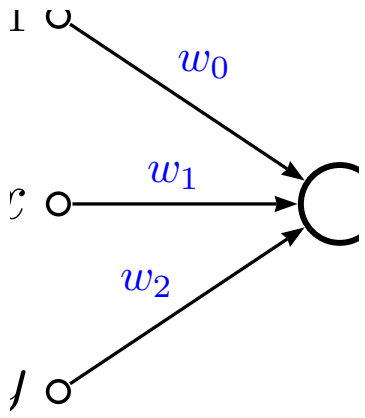
It is possible to define the bias and its corresponding signal differently. We can take the bias to equal θ (instead of $-\theta$) and the signal to equal -1 (instead of 1) because the net input does not change: $(-\theta) \times 1 = \theta \times (-1)$. Some textbooks take a bias equal to θ.



Linearly separable functions

How does a single node work with a binary threshold and bias?

Consider this case in the facing column. The switch of the output happens when the point (x, y) crosses the line $w_0 + w_1 x + w_2 y = 0$. In other words, this single-node network allows the **classification** of points in two categories, corresponding to the half-plane it lies in. In general, i.e., given more input signals, we say that this node realises a **linear classification** of inputs.

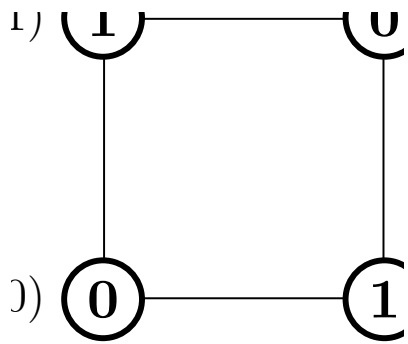


Non-linearity

Now, imagine the reverse situation where we do not know the weights, but we are given a set of points $\{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$ whose category is known, i.e., we know the network output for each point. Is it possible to find the input weights accordingly?

Since we assume here a binary threshold, it is clearly not possible in all cases. Consider the previous XOR example by drawing the four inputs as points (facing column) and putting the XOR values (**0** or **1**) inside the nodes.

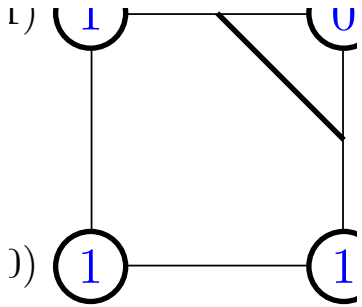
It is not possible to cut the figure with a line such as there are only **0**'s on one side and only **1**'s on the other: this is a **non-linear classification problem**.



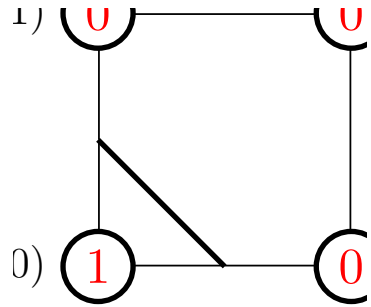
XOR revisited

If we consider the XOR network (page 8), we realise that the hidden layer transforms the non-linear problem into a linear one, i.e., the set of input vectors (before we called them points because there are only two variables) is mapped into a set that can be separated by a line corresponding to the boolean XOR function. Each node, H_1 and H_2, cuts the set by a different line and assigns 0 or 1 to each point, depending on their side:

Node H_1 mapping:

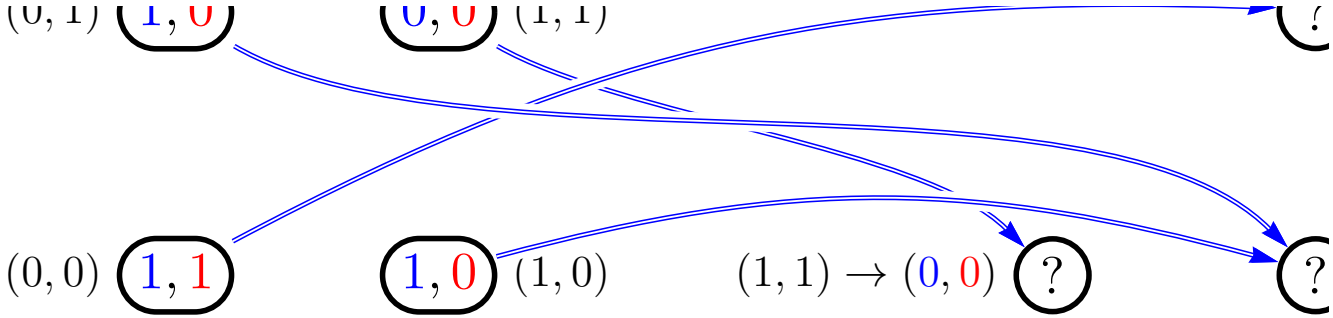


Node H_2 mapping:

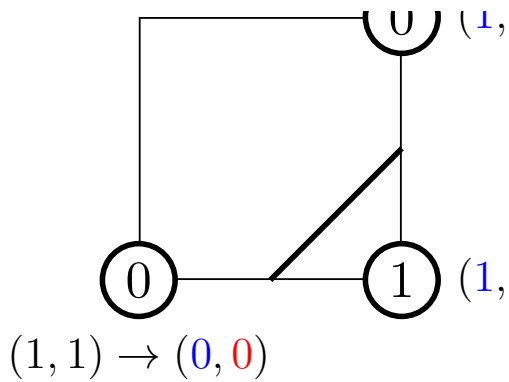


XOR revisited (cont)

The hidden layer transforms the four initial points in the following way.

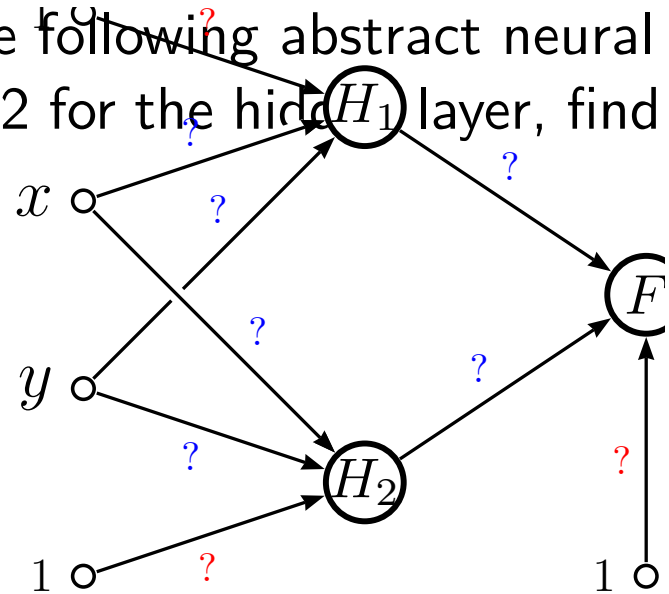


We put in the nodes the expected XOR values and the new set of points can be partitioned by a line: it is now a linear problem which can be solved by the single-node network $\{F\}$.



XOR revisited/Exercise

Exercise. Given the following abstract neural network and the mappings at page 12 for the hidden layer, find possible weights and bias.



Multi-layer networks

- With a single hidden layer, it is possible to represent any *continuous* function of the inputs with arbitrary accuracy.
- With two hidden ayers, even *discontinuous* functions can be represented.
- Unfortunately, given a network, it is very difficult to determine exactly which functions it can represent (by changing the weights) and which ones it can not.

Multi-layer networks (cont)

- Even designing a single hidden layer is difficult: the number of nodes (achieving the best classification) is hard to find.
- It is possible to get a linearly separable version of a given set of inputs by adding enough hidden layers, but this does not give good results in general (**over-fitting** problem).

Classification versus regression

The previous technique using lines to cut a set of points is used in linear problems (or which can be reduced to linear ones) for **classification**, i.e., assigning input data to one of a fixed number of possible classes (for instance, in character classification, a bit map representing the letter A is assigned the class A).

Another usage of abstract neural networks is **regression** problems, whose purpose is to predict the value of a *continuous* output variable (as opposed to a fixed number of classes).

The simplest regression is **linear regression**. In this case, a line is drawn in the set of points, such as it fits as much as possible all the points (in the classification problem, the line must separate the set, not try to pass through all the points).

Linear regression using least-squares fitting

One popular method for finding such fitting lines is the **least-squares method**.

It consists in defining a notion of **error** which is the difference between the ordinate of the given point and the ordinate of the point in the line with same abscissa. In other words, let $y = a + bx$ be the line equation. The error between (x_1, y_1) and the line is $\mid y_1 - (a + bx_1) \mid$.

The **total error** is the sum of the squares of all the individual errors:

$$\Pi(a, b) \triangleq \sum_{i=1}^{n} (y_i - (a + bx_i))^2$$

The most suitable line is the line which minimises $\Pi(a, b)$.

Linear regression using least-squares fitting (cont)

The minimum is reached when the partial derivatives equal zero:

$$\frac{\partial \Pi}{\partial a} = \frac{\partial \Pi}{\partial b} = 0$$

These constraints on a and b allow their determination in terms of the x_i and y_i.

Exercise. Give the exact formulæ for a and b.

So, following the statistics theory, networks for linear classification often implement the least-squares method. In case of a single-node network, this is straightforward since

$$(w_0, w_1, w_2) \triangleq (a, b, -1)$$

are possible weights for the network page 10.

Learning

It has been proved that any multi-layered network with linear activation functions is equivalent to a single-layer network with linear activation. Hence, in order to gain expressivity, non-linear functions, e.g. the sigmoid function, are necessary.

We showed a way to deduce the weights for the XOR network, but this was not a general method.

In more general cases,

- the network starts with **random weights** in a small interval and
- then is presented with all the inputs, several times (each sequence is called an **epoch**)
- and a special method adjusts the weights to **minimise the error**.

Learning (cont)

Each epoch should shorten the distance between the expected output and the one obtained (for each input vector), i.e., minimize the errors, until a **tolerance interval** (around the expected output) is reached.

The adaptative method must **converge**, i.e., that it exists a set of weights such as the errors for *all* the inputs lie in a tolerance interval.

In this case, the whole process is called **learning** or **supervised training**.

As a side-note, networks with back-edges (i.e.,, from upper to lower layer) can **oscillate**, i.e., not converge.

Back-propagation algorithm

One important learning is based on the **back-propagation algorithm**. The idea is as follows.

For each input vector, weight modifications are computed and propagated from the input layer to the output layer.

Then errors between expected and output values are propagated *back to the input layer*, modifying again the weights.

This algorithm is a bit involved and mathematically-oriented, so we shall skip it here.

Back-propagation algorithm/Applications

What about real applications of abstract neural networks?

Perhaps the most interesting feature of neural network is that, after the learning phase, they can induce an output for an unknown, real-valued, input vector, either in classification or regression problems.

For instance, it is possible to train some networks to recognise the decimal digits presented in a bit map. Then we can present a distorted (noisy) digit and the network will output the closest match, allowing thus **classification of unknown data**.

Another related application is **handwritten digit recognition** (see textbook section 20.7, page 752).

Bibliography


Ivan Bratko.
PROLOG – Programming for Artificial Intelligence.
Pearson Education and Addison Wesley, third edition, 2001.


Stuart J. Russell and Peter Norvig.
Artificial Intelligence: A Modern Approach.
Prentice Hall, second edition, dec 2002.
`http://aima.cs.berkeley.edu/`.


Leon Sterling and Ehud Shapiro.
The Art of Prolog.
Logic Programming. MIT Press, second edition, 1994.